

Riconoscimento dell'attività umana da smartphone

Progetto d'esame di Apprendimento Automatico e Apprendimento Profondo

Leonardo Angelini — matricola 0322500161

Docente: Prof.ssa Noemi Scarpato

Indice

Introduzione	2
Il dataset	2
Descrizione	2
Motivazioni della scelta	3
Esplorazione	3
Suddivisione dei dati	4
Il problema dei volontari	4
La suddivisione adottata	4
Standardizzazione	5
Analisi delle feature con PCA (punto 1)	5
Metodo	5
Varianza spiegata	5
Visualizzazione	6
Interpretazione delle componenti	6
Algoritmi di classificazione (punto 2)	6
Modelli scelti	6
Scelta degli iperparametri	7
Metriche e visualizzazioni (punti 3 e 4)	8
Metriche utilizzate	8
Logistic Regression	8
SVM (RBF)	9
Random Forest	10
Approccio di deep learning (punto 5a)	10
Architettura	10
Addestramento	11
Risultati	12
Confronto e discussione	12
Confronto complessivo	12
Errori ricorrenti	13
Deep learning e modelli shallow	13
Generalizzazione a nuove persone	14
Conclusioni	14
Appendice — Codice del notebook	15

Introduzione

Il progetto affronta un problema di **classificazione supervisionata multiclasse**: riconoscere l'attività che una persona sta svolgendo a partire dai segnali dei sensori inerziali di uno smartphone. Le attività da distinguere sono sei:

Classe	Attività
WALKING	camminare in piano
WALKING_UPSTAIRS	salire le scale
WALKING_DOWNSTAIRS	scendere le scale
SITTING	stare seduti
STANDING	stare in piedi
LAYING	stare sdraiati

Il lavoro segue i cinque punti della traccia:

1. analisi delle feature con la **PCA** e visualizzazione dei risultati;
2. implementazione di **tre algoritmi di classificazione** visti nel corso: Logistic Regression, Support Vector Machine e Random Forest;
3. calcolo di **precision, recall, F-measure, accuracy e ROC AUC**;
4. visualizzazione delle **matrici di confusione** e delle **curve ROC**;
5. opzione **a**: un approccio di **deep learning** (rete neurale MLP in PyTorch) addestrato sulla stessa suddivisione dei dati usata per i modelli shallow.

Tutto il codice è nel notebook `leonardo_angellini_0322500161.ipynb`, riportato in appendice. Le figure di questa relazione sono prodotte direttamente dal notebook.

Il dataset

Descrizione

È stato scelto il dataset **Human Activity Recognition Using Smartphones** dell'UCI Machine Learning Repository (id 240)¹. I dati sono stati raccolti da **30 volontari** tra i 19 e i 48 anni che hanno svolto le sei attività indossando in vita uno smartphone Samsung Galaxy S II. Accelerometro e giroscopio del telefono hanno registrato l'accelerazione lineare e la velocità angolare sui tre assi a 50 Hz.

I segnali sono stati filtrati e suddivisi in **finestre di 2,56 secondi** (128 campioni), sovrapposte al 50%. Da ogni finestra gli autori hanno calcolato **561 feature** nel dominio del tempo e della frequenza: medie, deviazioni standard, massimi e minimi, energia, entropia, coefficienti autoregressivi, correlazioni tra assi, frequenze dominanti e altre ancora. Ogni riga del dataset è quindi una finestra, descritta da 561 valori numerici ed etichettata con l'attività svolta.

¹<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Caratteristica	Valore
Righe (finestre)	10.299
Feature	561, tutte numeriche continue
Classi	6
Volontari	30
Valori mancanti	nessuno
Righe duplicate	nessuna
Intervallo dei valori	già normalizzati in $[-1, 1]$

Motivazioni della scelta

- **Adatto alla PCA.** Con 561 feature, molte delle quali fortemente correlate (ad esempio media e mediana dello stesso segnale, o lo stesso indicatore calcolato su assi diversi), la riduzione della dimensionalità ha un significato concreto e non è un semplice esercizio.
- **Adatto al deep learning.** Circa 10.000 esempi sono sufficienti per addestrare una rete neurale in modo sensato. Su dataset piccoli come Iris le reti rendono meno dei modelli shallow, come mostrato a lezione.
- **Problema multiclasse bilanciato**, in cui le metriche per classe e le curve ROC one-vs-rest raccontano qualcosa di più della sola accuracy.
- **Tema applicativo concreto**, con applicazioni in sanità, fitness e assistenza agli anziani.
- **Non ancora usato** dagli altri studenti nel repository del corso.

Esplorazione

Il dataset non ha valori mancanti né righe duplicate, e le feature sono già scalate nell'intervallo $[-1, 1]$. Le sei classi sono **abbastanza bilanciate**: si va dal 13,7% di WALKING_DOWNSTAIRS al 18,9% di LAYING (Figura 1). L'accuracy resta quindi una metrica significativa. Accanto a essa riportiamo comunque le metriche *macro*, cioè la media non pesata sulle classi, che trattano allo stesso modo tutte le attività.

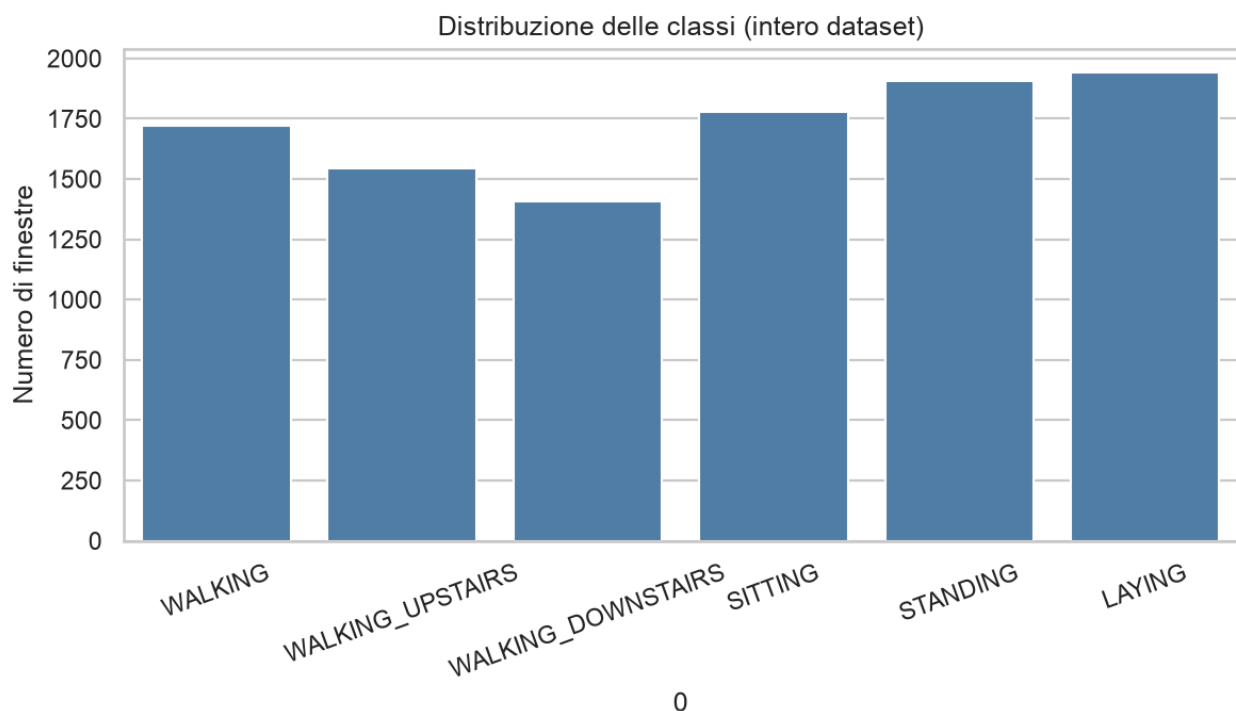


Figura 1: Distribuzione delle classi nell'intero dataset

Il dataset non è importabile con la libreria `ucimlrepo`: il notebook scarica automaticamente lo zip ufficiale da UCI alla prima esecuzione. Nel file originale 42 nomi di feature sono duplicati; sono stati resi univoci aggiungendo un suffisso numerico.

Suddivisione dei dati

Il problema dei volontari

La lezione sulla gestione dei progetti raccomanda di dividere i dati in training, validation e test set (proporzione indicativa 70/20/10), in modo stratificato e riproducibile. In questo dataset c'è però una particolarità di cui tenere conto: **le finestre dello stesso volontario si somigliano molto**, perché ognuno ha il proprio modo di camminare, sedersi e muoversi.

Una divisione casuale riga per riga metterebbe le finestre della stessa persona sia nel training sia nel test. Il modello verrebbe così valutato su persone che ha già «visto», e le metriche sarebbero ottimistiche rispetto all'uso reale, in cui l'app deve funzionare su utenti nuovi. Per questo la divisione è stata fatta **per volontario**.

La suddivisione adottata

- **Test set:** la partizione ufficiale del dataset, con 9 volontari mai usati in addestramento. È il caso, citato a lezione, dei benchmark pubblici con partizioni predefinite per consentire confronti obiettivi tra algoritmi.
- **Validation set:** ricavato dal training ufficiale tenendo da parte circa il 20% dei volontari con `GroupShuffleSplit` (`random_state=42`). Anche la scelta degli iperparametri avviene così su persone nuove.
- **Training set:** i volontari restanti.

Insieme	Righe	Quota	Volontari
Training	5.551	53,9%	16
Validation	1.801	17,5%	5
Test	2.947	28,6%	9

Le proporzioni si discostano dal 70/20/10 indicativo perché sono vincolate alla partizione ufficiale. In cambio si ottiene una valutazione più onesta e confrontabile con la letteratura.

La divisione per volontario rende impossibile una stratificazione riga per riga, ma **la distribuzione delle classi resta simile nei tre insiemi**, perché ogni volontario ha svolto tutte e sei le attività:

Classe	Training	Validation	Test
WALKING	16,0%	18,8%	16,8%
WALKING_UPSTAIRS	14,4%	15,3%	16,0%
WALKING_DOWNSTAIRS	13,4%	13,4%	14,3%
SITTING	17,9%	16,3%	16,7%
STANDING	19,0%	17,8%	18,1%
LAYING	19,4%	18,4%	18,2%

La stessa suddivisione è usata per tutti i modelli, compresa la rete neurale, come richiesto dalla traccia.

Standardizzazione

Le feature sono state standardizzate con `StandardScaler` (media 0, deviazione standard 1). Lo scaler viene stimato **solo sul training set** e poi applicato a validation e test, così nessuna informazione dei dati di valutazione entra nell'addestramento (*data leakage*). Per lo stesso motivo anche la PCA è stimata solo sul training set.

Analisi delle feature con PCA (punto 1)

Metodo

La **Principal Component Analysis** trasforma le 561 feature correlate in un nuovo insieme di variabili non correlate, le componenti principali. Ogni componente è una combinazione lineare delle feature originali, e le componenti sono ordinate per quantità di varianza spiegata. Seguendo la lezione sulla PCA:

- i dati sono centrati e scalati con lo `StandardScaler`;
- si calcola l'**Explained Variance Ratio** individuale e cumulativa, per capire quante componenti servono a mantenere il 90–95% della varianza;
- si proiettano i dati sulle prime due e tre componenti per visualizzarli;
- si analizzano i **loadings**, cioè il peso di ogni feature originale nelle componenti, per interpretarle.

Varianza spiegata

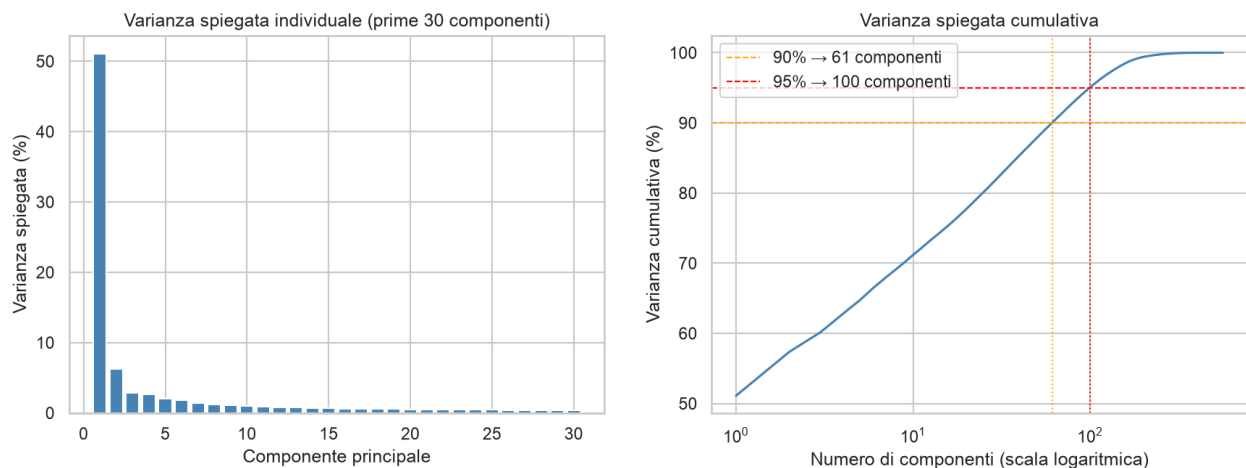


Figura 2: Varianza spiegata individuale e cumulativa

- **PC1 da sola spiega il 51,1%** della varianza totale, PC2 il 6,3% e PC3 il 2,8%.
- Per il **90%** della varianza servono **61 componenti**, per il **95%** ne servono **100** e per il 99% ne servono 176, su 561 feature.

C'è quindi una direzione dominante, ma il resto dell'informazione è distribuito su molte componenti. Le feature originali sono molto ridondanti: 100 componenti bastano a conservare il 95% dell'informazione di 561 variabili.

Visualizzazione

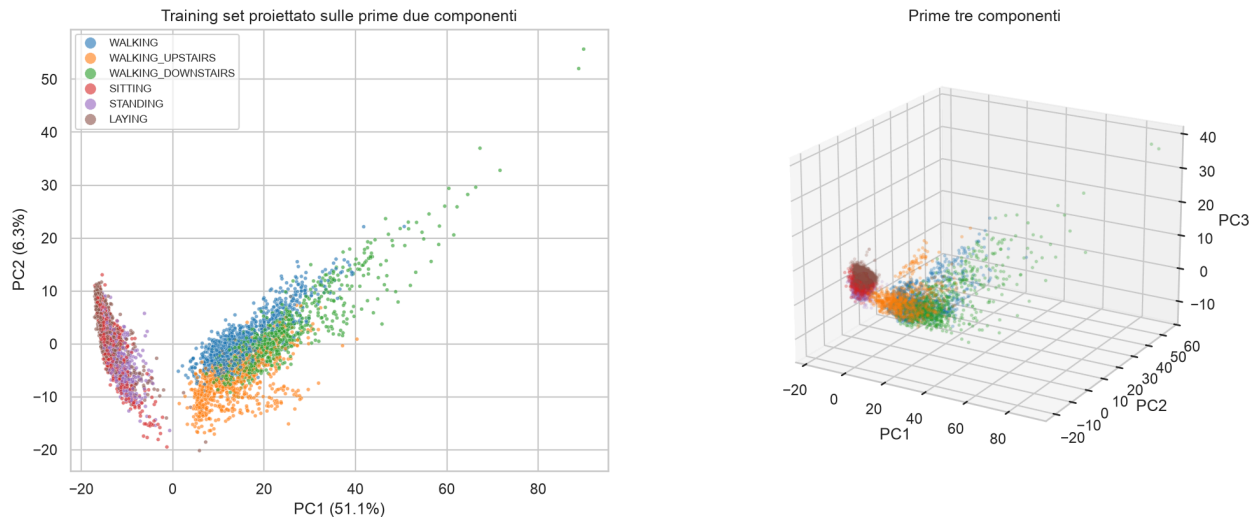


Figura 3: Training set proiettato sulle prime due e tre componenti principali

La proiezione mostra due gruppi ben distinti:

- a sinistra le **attività statiche** (seduto, in piedi, sdraiato);
- a destra le **attività dinamiche** (le tre camminate).

Interpretazione delle componenti

Le feature con peso maggiore su **PC1** sono le *sma* (*signal magnitude area*) e le magnitudini del *jerk*, la derivata dell'accelerazione, di accelerometro e giroscopio. Sono tutte misure di **quanto è intenso il movimento**: PC1 è in pratica un indice di attività fisica, ed è per questo che separa così nettamente le attività statiche da quelle dinamiche.

Su **PC2** pesano soprattutto i coefficienti autoregressivi e la frequenza media dei segnali, cioè il **ritmo e la forma** del movimento. PC2 aiuta a distinguere le camminate tra loro: salire le scale si concentra nella parte bassa del grafico, scendere le scale in quella alta e più dispersa.

Le tre attività statiche invece **si sovrappongono quasi completamente** nelle prime componenti. È un primo segnale di quello che confermeranno le matrici di confusione: seduto e in piedi sono le classi più difficili da separare.

Conclusione sulla PCA. Due o tre componenti bastano a visualizzare la struttura principale dei dati, ma non a classificarli bene. I classificatori sono stati quindi addestrati su tutte le 561 feature standardizzate; la PCA qui è uno strumento di analisi e visualizzazione.

Algoritmi di classificazione (punto 2)

Modelli scelti

Tra gli algoritmi del corso sono stati scelti tre modelli con principi di funzionamento diversi, per confrontare approcci lineari, a margine e basati su alberi:

Modello	Famiglia	Motivazione
Logistic Regression	lineare, probabilistico	baseline semplice e interpretabile; con 561 feature standardizzate le classi potrebbero essere già quasi separabili linearmente
SVM con kernel RBF	a margine massimo, non lineare	molto efficace negli spazi ad alta dimensionalità
Random Forest	ensemble di alberi decisionali	cattura interazioni non lineari tra le feature ed è robusta al rumore

Scelta degli iperparametri

Per ogni modello sono state provate poche configurazioni, scelte tra i valori più comuni. Ognuna è stata addestrata sul training set e valutata con la **F1 macro sul validation set**. Il test set non è mai stato usato in questa fase, come raccomandato a lezione, altrimenti le metriche finali sarebbero falsamente ottimistiche.

Logistic Regression: C è l'inverso della forza della regolarizzazione.

C	0,01	0,1	1	10
F1 macro (validation)	0,950	0,957	0,953	0,947

SVM (RBF): C è la penalità sugli errori, gamma è l'ampiezza del kernel.

C	gamma = «scale»	gamma = 0,0001
1	0,943	0,923
10	0,945	0,953
100	0,943	0,953

Random Forest (300 alberi): max_depth è la profondità massima, max_features la quota di feature considerata a ogni split.

max_depth	max_features = «sqrt»	max_features = 0,1
nessun limite	0,926	0,938
15	0,924	0,936

Le configurazioni scelte sono: Logistic Regression con **C = 0,1**, SVM con **C = 10 e gamma = 0,0001**, Random Forest con **profondità libera e max_features = 0,1**.

Per la Logistic Regression vince una regolarizzazione piuttosto forte (C = 0,1): con tante feature correlate, limitare i coefficienti aiuta a generalizzare verso volontari nuovi. Per la Random Forest, considerare il 10% delle feature a ogni split (circa 56) funziona meglio della radice quadrata (circa 24): con molte feature ridondanti, guardarne di più aumenta la probabilità di trovare quelle davvero informative.

La SVM non produce probabilità ma solo distanze dal margine. Per calcolare la ROC AUC è stato attivato probability=True, che le stima con la calibrazione di Platt tramite una cross-validation interna sul solo training set.

Metriche e visualizzazioni (punti 3 e 4)

Metriche utilizzate

Per ogni modello sono state calcolate sul **test set**:

- **Accuracy**, la quota di predizioni corrette sul totale;
- **Precision**, la quota di finestre assegnate a una classe che appartengono davvero a quella classe;
- **Recall**, la quota di finestre di una classe riconosciute correttamente;
- **F-measure (F1)**, la media armonica di precision e recall;
- **ROC AUC**, l'area sotto la curva ROC.

Trattandosi di un problema multiclasse, precision, recall e F1 sono riportate **per classe** e come **media macro**. La ROC AUC è calcolata in modalità **one-vs-rest**: per ogni attività si considera il problema binario «questa attività contro tutte le altre» e si fa la media delle sei AUC. Nelle figure, accanto alle sei curve ROC, è riportata la curva **micro-average**, come nell'esempio multiclasse visto a lezione. Poiché tutte le curve sono schiacciate contro l'angolo in alto a sinistra, un riquadro ingrandito mostra la zona con False Positive Rate tra 0 e 0,15.

Nelle matrici di confusione le righe sono le classi reali e le colonne le classi predette: i valori fuori dalla diagonale sono gli errori.

Logistic Regression

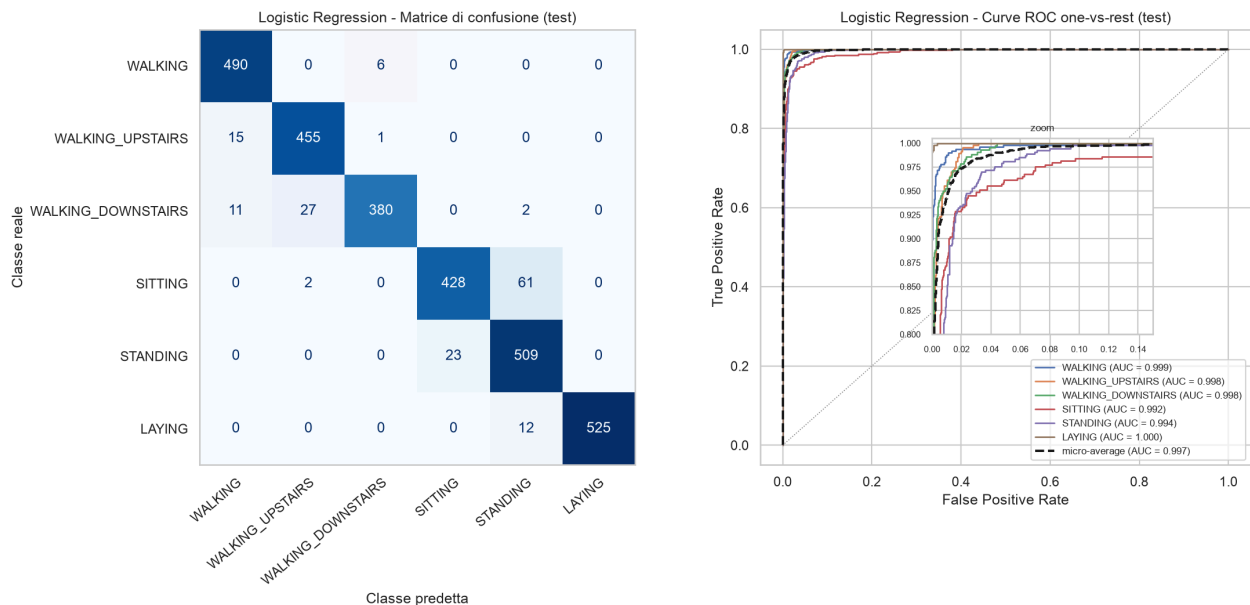


Figura 4: Logistic Regression: matrice di confusione e curve ROC sul test set

Classe	Precision	Recall	F1
WALKING	0,950	0,988	0,968
WALKING_UPSTAIRS	0,940	0,966	0,953
WALKING_DOWNSTAIRS	0,982	0,905	0,942
SITTING	0,949	0,872	0,909
STANDING	0,872	0,957	0,912
LAYING	1,000	0,978	0,989
Media macro	0,949	0,944	0,945

Accuracy **94,6%**, ROC AUC macro **0,997**.

L'errore principale riguarda **61 finestre di SITTING classificate come STANDING** (e 23 nel verso opposto), che abbassano la recall di SITTING a 0,872. Tra le camminate, 27 discese sono scambiate per salite. Un dettaglio interessante: la Logistic Regression è l'unico modello che sbaglia in modo apprezzabile su LAYING, con 12 finestre classificate come STANDING (gli altri modelli ne sbagliano al massimo una).

SVM (RBF)

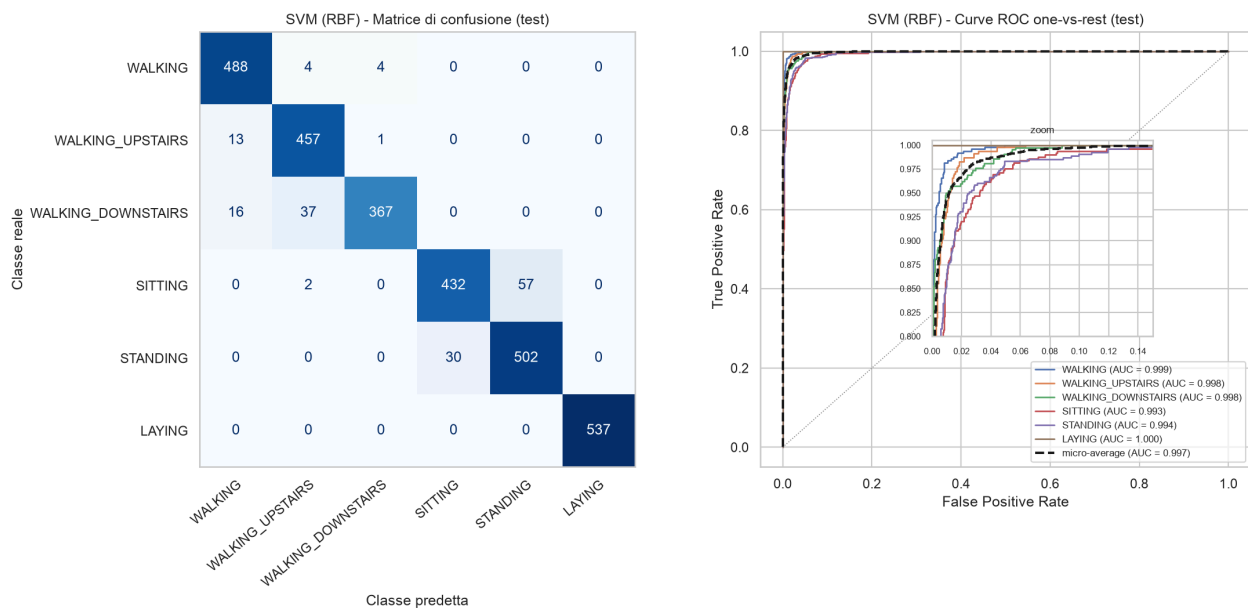


Figura 5: SVM: matrice di confusione e curve ROC sul test set

Classe	Precision	Recall	F1
WALKING	0,944	0,984	0,963
WALKING_UPSTAIRS	0,914	0,970	0,941
WALKING_DOWNSTAIRS	0,987	0,874	0,927
SITTING	0,935	0,880	0,907
STANDING	0,898	0,944	0,920
LAYING	1,000	1,000	1,000
Media macro	0,946	0,942	0,943

Accuracy **94,4%**, ROC AUC macro **0,997**.

La SVM riconosce **LAYING senza nessun errore**. Gli errori tra seduto e in piedi sono simili a quelli della Logistic Regression (57 e 30), mentre la discesa delle scale è confusa più spesso: 37 finestre scambiate per salita e 16 per camminata in piano.

Random Forest

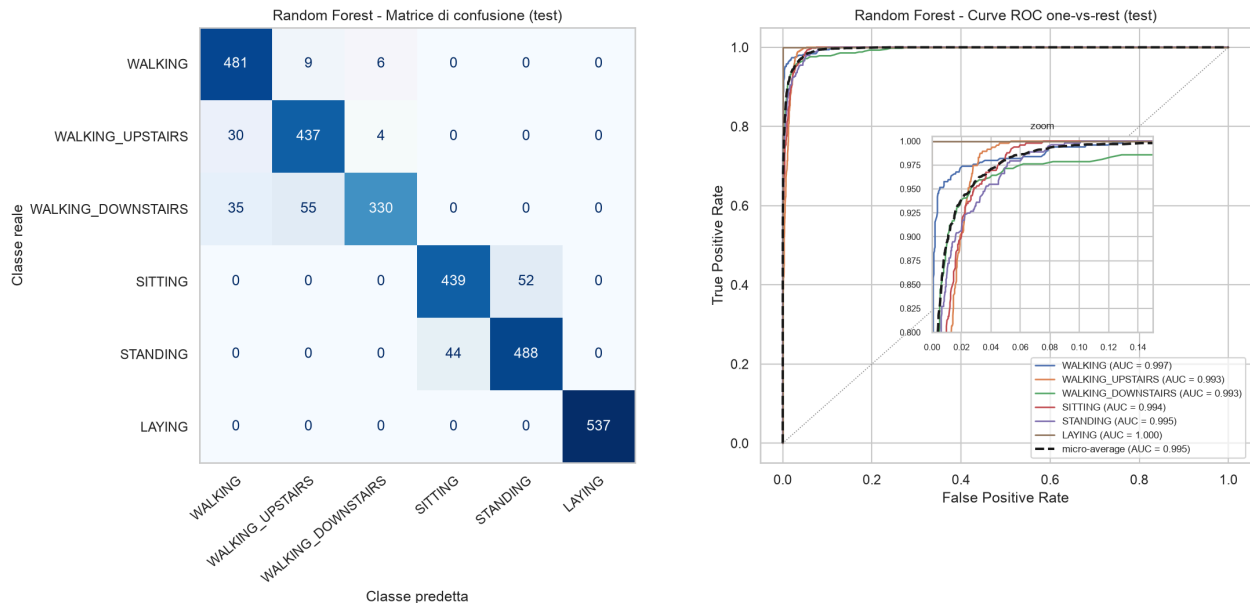


Figura 6: Random Forest: matrice di confusione e curve ROC sul test set

Classe	Precision	Recall	F1
WALKING	0,881	0,970	0,923
WALKING_UPSTAIRS	0,872	0,928	0,899
WALKING_DOWNSTAIRS	0,971	0,786	0,868
SITTING	0,909	0,894	0,901
STANDING	0,904	0,917	0,910
LAYING	1,000	1,000	1,000
Media macro	0,923	0,916	0,917

Accuracy **92,0%**, ROC AUC macro **0,995**.

La Random Forest è il modello più debole, soprattutto sulle **camminate**: 90 discese su 420 sono classificate male (55 come salita e 35 come camminata in piano), e la recall di WALKING_DOWNSTAIRS scende a 0,786. Anche la confusione tra seduto e in piedi è la più equilibrata e la più alta nel totale (52 + 44). Una possibile spiegazione è che ogni split usa una sola feature alla volta, mentre la distinzione tra camminate dipende da combinazioni di molte feature, che i modelli lineari e la SVM gestiscono meglio.

Approccio di deep learning (punto 5a)

Architettura

È stata implementata una rete neurale **MLP (Multi-Layer Perceptron)** in PyTorch, sul modello del classificatore multiclasse presentato nella lezione sulla classificazione con reti neurali:

Strato	Dimensione	Dettagli
Input	561	le stesse feature standardizzate dei modelli shallow
Nascosto 1	256	Linear, BatchNorm, ReLU, Dropout 0,3
Nascosto 2	128	Linear, BatchNorm, ReLU, Dropout 0,3
Output	6	un logit per classe

La rete ha **178.310 parametri** addestrabili. Scelte principali:

- **funzione di perdita:** CrossEntropyLoss, che incorpora la softmax (i logit non vengono passati alla softmax prima della loss, per stabilità numerica);
- **ottimizzatore:** Adam con learning rate 0,001 e batch da 64 esempi;
- **regolarizzazione:** Dropout e BatchNorm contro l'overfitting;
- **early stopping:** a ogni epoca si misura la loss sul validation set; l'addestramento si ferma dopo 10 epoche consecutive senza miglioramenti e si ripristinano i pesi dell'epoca migliore.

Come richiesto dalla traccia, la rete usa **esattamente gli stessi training, validation e test set** dei modelli shallow. Il validation set è quello descritto nella sezione sulla suddivisione dei dati: non è stato usato `validation_split` né un'altra divisione interna. I seed sono fissati, quindi i risultati sono riproducibili.

Addestramento

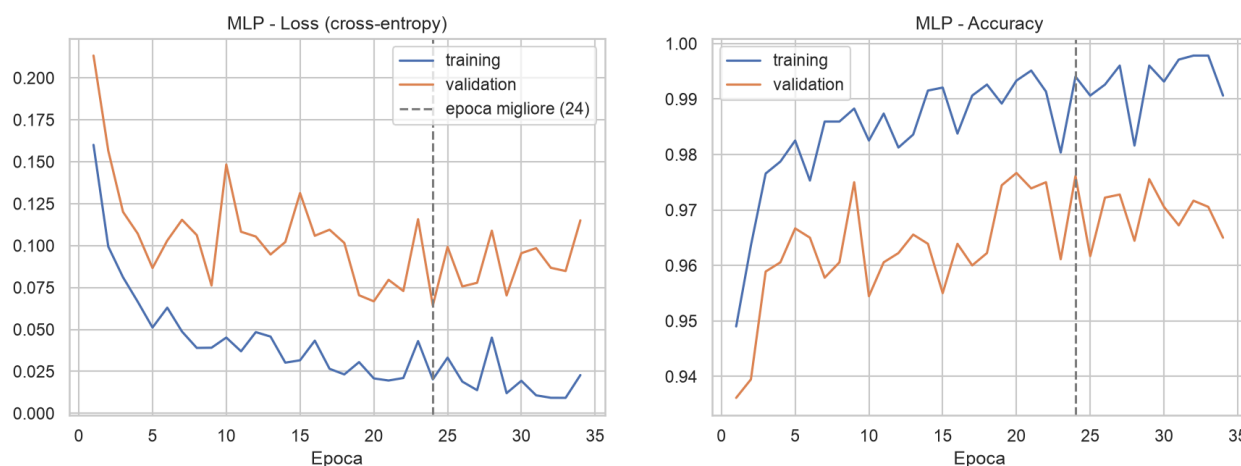


Figura 7: Curve di loss e accuracy della rete MLP su training e validation

La loss di validazione ha raggiunto il minimo (0,064) all'**epoca 24**; l'addestramento si è fermato all'epoca 34 per l'early stopping, ripristinando i pesi dell'epoca 24.

Le curve mostrano un **leggero overfitting**: l'accuracy di training arriva vicino al 99%, quella di validazione si ferma intorno al 97%. Dropout, BatchNorm ed early stopping lo tengono sotto controllo. La loss di validazione oscilla parecchio da un'epoca all'altra anche perché il validation set è piccolo (5 volontari): basta che cambino le predizioni su poche finestre di una persona per spostare la media.

Risultati

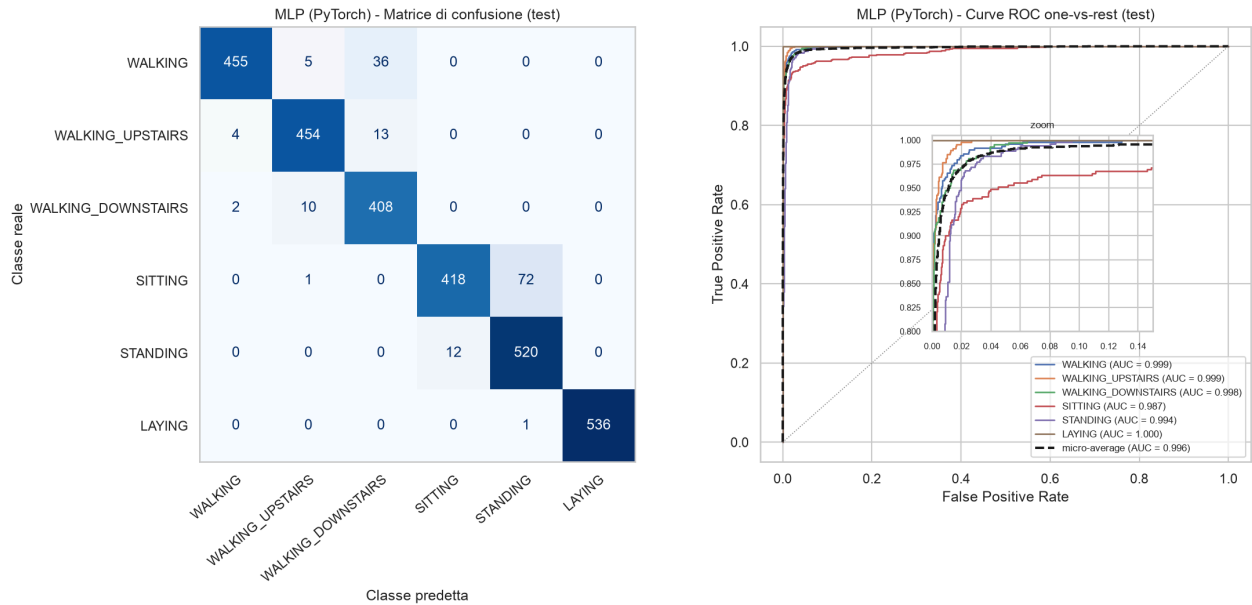


Figura 8: Rete MLP: matrice di confusione e curve ROC sul test set

Classe	Precision	Recall	F1
WALKING	0,987	0,917	0,951
WALKING_UPSTAIRS	0,966	0,964	0,965
WALKING_DOWNSTAIRS	0,893	0,971	0,930
SITTING	0,972	0,851	0,908
STANDING	0,877	0,977	0,924
LAYING	1,000	0,998	0,999
Media macro	0,949	0,947	0,946

Accuracy **94,7%**, ROC AUC macro **0,996**.

La MLP commette errori **diversi** da quelli dei modelli shallow. Riconosce molto bene la discesa delle scale (recall 0,971 contro 0,79–0,91 degli altri modelli), ma in cambio **classifica come discesa 36 camminate in piano**. Tra le attività statiche tende a spostare le finestre verso STANDING: 72 SITTING sono predette come STANDING, ma solo 12 nel verso opposto.

Confronto e discussione

Confronto complessivo

Modello	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	ROC AUC (macro)
MLP (PyTorch)	0,947	0,949	0,947	0,946	0,996
Logistic Regression	0,946	0,949	0,944	0,945	0,997
SVM (RBF)	0,944	0,946	0,942	0,943	0,997
Random Forest	0,920	0,923	0,916	0,917	0,995

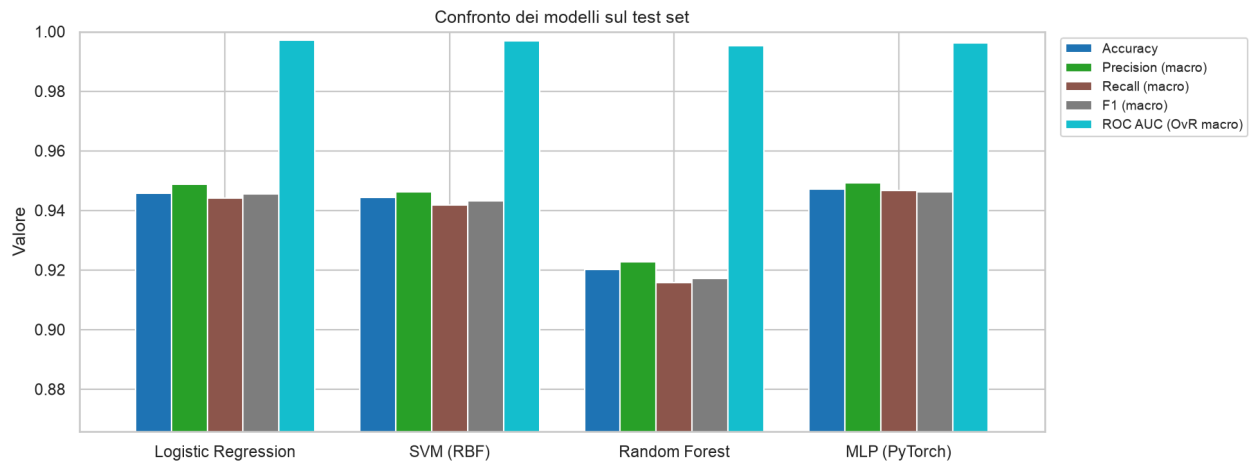


Figura 9: Confronto delle metriche dei quattro modelli sul test set

Tutti i modelli superano il 92% di accuracy su volontari mai visti, e la ROC AUC è sopra 0,99 per tutti. L'AUC è quasi identica tra i modelli anche quando l'accuracy differisce di alcuni punti. Il motivo è che l'AUC misura quanto il modello **ordina** bene gli esempi (le finestre di una classe ricevono punteggi più alti delle altre), indipendentemente dalla soglia di decisione. L'accuracy invece dipende dalla decisione finale: quando due classi si sovrappongono, come seduto e in piedi, il modello può ordinarle abbastanza bene ma sbagliare comunque la classe esatta.

Errori ricorrenti

Le matrici di confusione mostrano gli stessi punti deboli per tutti i modelli.

1. **Seduto e in piedi si confondono sempre**, con 84–96 finestre scambiate a seconda del modello, e SITTING ha la recall più bassa per tutti (0,85–0,89). Con il telefono in vita le due posture producono segnali quasi identici: il corpo è fermo e la gravità agisce nella stessa direzione rispetto al telefono. È la stessa sovrapposizione già vista nella PCA.
2. **Le camminate si confondono tra loro**, ma in modo diverso a seconda del modello. I modelli shallow scambiano la discesa per salita o camminata in piano; la MLP fa il contrario e scambia la camminata in piano per discesa. Il numero di errori complessivo è simile, la loro distribuzione no: è un'informazione che la sola accuracy non mostra.
3. **LAYING è quasi sempre riconosciuta** (recall tra 0,978 e 1,000): da sdraiati la gravità cambia asse rispetto al telefono, e il segnale diventa inconfondibile.

Deep learning e modelli shallow

La rete MLP ottiene il risultato migliore, ma il vantaggio sulla Logistic Regression è di circa **0,1 punti percentuali**, cioè **4 finestre su 2.947**. È una differenza troppo piccola per dire che un modello è migliore dell'altro: con un seed diverso o un'altra suddivisione dei volontari l'ordine potrebbe benissimo invertirsi.

Una spiegazione è che le 561 feature del dataset sono già state **progettate a mano** da esperti di elaborazione dei segnali. Contengono già l'informazione utile, e un modello lineare ben regolarizzato riesce a sfruttarla quasi del tutto. Il vero vantaggio del deep learning, cioè apprendere da solo le rappresentazioni dai dati invece di richiedere un'ingegnerizzazione manuale delle feature, emergerebbe meglio partendo dai **segnali inerziali grezzi** inclusi nel dataset (cartella Inertial Signals), con reti convoluzionali o ricorrenti. È un possibile sviluppo futuro di questo lavoro.

A parità di risultato, la Logistic Regression resta la scelta più conveniente in pratica: si addestra in pochi secondi, è più semplice da mettere in produzione su uno smartphone ed è più interpretabile.

Generalizzazione a nuove persone

Passando dal validation al test set le prestazioni calano leggermente: per la Logistic Regression la F1 macro scende da 0,957 a 0,945. Conferma che la vera difficoltà del problema è **generalizzare a persone nuove**, ed è il motivo per cui la divisione dei dati per volontario, e non per singola riga, era necessaria. Con una divisione casuale riga per riga le metriche sarebbero state più alte, ma meno rappresentative dell'uso reale.

Conclusioni

Il progetto ha affrontato il riconoscimento di sei attività umane dai sensori di uno smartphone, seguendo tutti i punti della traccia:

- la **PCA** ha mostrato che le 561 feature sono molto ridondanti (100 componenti conservano il 95% della varianza) e che la prima componente, legata all'intensità del movimento, separa nettamente le attività statiche da quelle dinamiche;
- tra i **tre classificatori shallow**, Logistic Regression (94,6%) e SVM (94,4%) si equivalgono, mentre la Random Forest resta indietro (92,0%), soprattutto sulle camminate;
- la **rete MLP**, addestrata sulla stessa suddivisione dei dati, ottiene il risultato migliore (94,7%), ma con un vantaggio non significativo rispetto alla Logistic Regression;
- per tutti i modelli la difficoltà principale è distinguere **seduto da in piedi**, due posture che il telefono in vita percepisce in modo quasi identico.

Limiti.

- I risultati si riferiscono a una sola suddivisione dei dati e a un solo seed. Una stima più robusta richiederebbe una cross-validation per volontario (ad esempio GroupKFold).
- La ricerca degli iperparametri è limitata a poche configurazioni per modello.

Possibili sviluppi.

- Addestrare reti convoluzionali o ricorrenti sui segnali grezzi.
- Aggiungere feature specifiche per distinguere seduto da in piedi, ad esempio l'inclinazione del telefono rispetto alla gravità.
- Applicare tecniche di spiegabilità come SHAP per capire quali feature guidano le decisioni dei modelli.

Uso di strumenti di intelligenza artificiale. Nello sviluppo del codice e nella stesura della relazione è stato usato un assistente AI come strumento di supporto; tutte le scelte progettuali e i risultati sono stati verificati e compresi dall'autore.

Appendice — Codice del notebook

Di seguito il codice completo del notebook leonardo_angelini_0322500161.ipynb, nell'ordine di esecuzione. Le celle di testo e gli output sono omessi: sono riportati e commentati nelle sezioni precedenti.

```
import os
import random
import warnings
import urllib.request
import zipfile
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.decomposition import PCA
from sklearn.model_selection import GroupShuffleSplit
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, roc_curve, auc, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

sns.set_theme(style="whitegrid")
OUT = Path("outputs")
OUT.mkdir(exist_ok=True)

def salva(nome):
    # Salva la figura corrente in outputs/ (usata poi nella relazione)
    plt.savefig(OUT / f"{nome}.png", dpi=150, bbox_inches="tight")

DATA_DIR = Path("data")
HAR_DIR = DATA_DIR / "UCI HAR Dataset"
URL = "https://archive.ics.uci.edu/static/public/240/human+activity+recognition+using+smartphones.zip"

if not HAR_DIR.exists():
    DATA_DIR.mkdir(exist_ok=True)
    zip_esterno = DATA_DIR / "har.zip"
    urllib.request.urlretrieve(URL, zip_esterno)
    with zipfile.ZipFile(zip_esterno) as z:
```

```

        z.extractall(DATA_DIR)
# Lo zip UCI contiene a sua volta lo zip con i dati
with zipfile.ZipFile(DATA_DIR / "UCI HAR Dataset.zip") as z:
    z.extractall(DATA_DIR)

# Nomi delle feature: 42 nomi sono duplicati nel file originale, li rendiamo univoci
nomi = pd.read_csv(HAR_DIR / "features.txt", sep=r"\s+", header=None, names=["idx",
"nome"])[["nome"]]
conteggio = nomi.groupby(nomi).cumcount()
feature_names = np.where(conteggio > 0, nomi + "_" + conteggio.astype(str), nomi)

etichette = pd.read_csv(HAR_DIR / "activity_labels.txt", sep=r"\s+", header=None,
names=["id", "attivita"])
CLASSI = etichette["attivita"].tolist()

def carica(parte):
    X = pd.read_csv(HAR_DIR / parte / f"X_{parte}.txt", sep=r"\s+", header=None,
names=feature_names)
    y = pd.read_csv(HAR_DIR / parte / f"y_{parte}.txt", header=None)[0] - 1 #
classi 0..5
    soggetti = pd.read_csv(HAR_DIR / parte / f"subject_{parte}.txt", header=None)[0]
    return X, y, soggetti

X_train_full, y_train_full, sogg_train_full = carica("train")
X_test, y_test, sogg_test = carica("test")

print("Train ufficiale:", X_train_full.shape, "- volontari:",
sorted(sogg_train_full.unique().tolist()))
print("Test ufficiale: ", X_test.shape, "- volontari:",
sorted(sogg_test.unique().tolist()))
print("Classi:", CLASSI)

X_tutto = pd.concat([X_train_full, X_test])
y_tutto = pd.concat([y_train_full, y_test])

print("Valori mancanti:", X_tutto.isna().sum().sum())
print("Righe duplicate:", X_tutto.duplicated().sum())
print(f"Intervallo dei valori: [{X_tutto.values.min():.2f},
{X_tutto.values.max():.2f}]")
X_tutto.iloc[:, :8].describe().round(3)
distribuzione = y_tutto.map(dict(enumerate(CLASSI))).value_counts().reindex(CLASSI)

plt.figure(figsize=(9, 4))
sns.barplot(x=distribuzione.index, y=distribuzione.values, color="steelblue")
plt.title("Distribuzione delle classi (intero dataset)")
plt.ylabel("Numero di finestre")
plt.xticks(rotation=20)
salva("01_distribuzione_classi")
plt.show()

(distribuzione / distribuzione.sum() * 100).round(1).rename("%")

```



```

gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=SEED)
idx_train, idx_val = next(gss.split(X_train_full, y_train_full, groups=sogg_train_full))

X_train, y_train = X_train_full.iloc[idx_train], y_train_full.iloc[idx_train]
X_val, y_val = X_train_full.iloc[idx_val], y_train_full.iloc[idx_val]

totale = len(X_train_full) + len(X_test)
for nome, X_, s_ in [("Training", X_train, sogg_train_full.iloc[idx_train]),
                    ("Validation", X_val, sogg_train_full.iloc[idx_val]),
                    ("Test", X_test, sogg_test)]:
    print(f"{nome:<11} {len(X_):>5} righe ({len(X_) / totale:5.1%}) - {s_.nunique()}
volontari: {sorted(s_.unique().tolist())}")

# Verifica: la distribuzione delle classi è simile nei tre insiemi
pd.DataFrame({
    "train %": y_train.value_counts(normalize=True).sort_index() * 100,
    "val %": y_val.value_counts(normalize=True).sort_index() * 100,
    "test %": y_test.value_counts(normalize=True).sort_index() * 100,
}).set_axis(CLASSI).round(1)

scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_val_s = scaler.transform(X_val)
X_test_s = scaler.transform(X_test)

y_train, y_val, y_test = y_train.to_numpy(), y_val.to_numpy(), y_test.to_numpy()
print(X_train_s.shape, X_val_s.shape, X_test_s.shape)

pca_full = PCA(random_state=SEED).fit(X_train_s)
evr = pca_full.explained_variance_ratio_
cumulata = np.cumsum(evr)

n90 = int(np.argmax(cumulata >= 0.90)) + 1
n95 = int(np.argmax(cumulata >= 0.95)) + 1
n99 = int(np.argmax(cumulata >= 0.99)) + 1
print(f"PC1 spiega il {evr[0]:.1%} della varianza, PC2 il {evr[1]:.1%}, PC3 il
{evr[2]:.1%}")
print(f"Componenti per il 90% della varianza: {n90}")
print(f"Componenti per il 95% della varianza: {n95}")
print(f"Componenti per il 99% della varianza: {n99} (su 561 feature)")

fig, ax = plt.subplots(1, 2, figsize=(14, 4.5))
ax[0].bar(range(1, 31), evr[:30] * 100, color="steelblue")
ax[0].set_title("Varianza spiegata individuale (prime 30 componenti)")
ax[0].set_xlabel("Componente principale")
ax[0].set_ylabel("Varianza spiegata (%)")

ax[1].plot(range(1, len(cumulata) + 1), cumulata * 100, color="steelblue")
for soglia, n, colore in [(90, n90, "orange"), (95, n95, "red")]:
    ax[1].axhline(soglia, ls="--", color=colore, lw=1, label=f"{soglia}% → {n}
componenti")
    ax[1].axvline(n, ls=":", color=colore, lw=1)
ax[1].set_xscale("log")
ax[1].set_title("Varianza spiegata cumulativa")
ax[1].set_xlabel("Numero di componenti (scala logaritmica)")

```

```

ax[1].set_ylabel("Varianza cumulativa (%)")
ax[1].legend()
salva("02_pca_varianza_spiegata")
plt.show()

Z_train = pca_full.transform(X_train_s)[: , :3]
df_pca = pd.DataFrame(Z_train, columns=["PC1", "PC2", "PC3"])
df_pca["attivit "] = pd.Categorical([CLASSI[i] for i in y_train], categories=CLASSI)

fig = plt.figure(figsize=(16, 6))
ax1 = fig.add_subplot(1, 2, 1)
sns.scatterplot(data=df_pca, x="PC1", y="PC2", hue="attivit ", s=8, alpha=0.6,
palette="tab10", ax=ax1)
ax1.set_title("Training set proiettato sulle prime due componenti")
ax1.set_xlabel(f"PC1 ({evr[0]:.1%})")
ax1.set_ylabel(f"PC2 ({evr[1]:.1%})")
ax1.legend(markerscale=3, fontsize=8)

ax2 = fig.add_subplot(1, 2, 2, projection="3d")
colori = sns.color_palette("tab10", len(CLASSI))
for k, nome in enumerate(CLASSI):
    m = y_train == k
    ax2.scatter(Z_train[m, 0], Z_train[m, 1], Z_train[m, 2], s=3, alpha=0.5,
color=colori[k], label=nome)
ax2.set_title("Prime tre componenti")
ax2.set_xlabel("PC1")
ax2.set_ylabel("PC2")
ax2.set_zlabel("PC3")
ax2.view_init(elev=20, azim=-60)
salva("03_pca_scatter")
plt.show()

loadings = pd.DataFrame(pca_full.components_[:2].T, index=feature_names, columns=["PC1",
"PC2"])
for pc in ["PC1", "PC2"]:
    top = loadings[pc].abs().sort_values(ascending=False).head(8).index
    print(f"\nFeature con peso maggiore su {pc}:")
    print(loadings.loc[top, pc].round(3).to_string())

def scegli_iperparametri(nome, crea_modello, griglia):
    risultati = []
    for params in griglia:
        modello = crea_modello(**params).fit(X_train_s, y_train)
        f1_val = f1_score(y_val, modello.predict(X_val_s), average="macro")
        risultati.append(**params, "F1 macro (val)": round(f1_val, 4))
    tabella = pd.DataFrame(risultati)
    migliore = griglia[int(tabella["F1 macro (val)"].idxmax())]
    print(f"{nome}: migliori iperparametri {migliore}")
    return migliore, tabella

griglia_lr = [{"C": c} for c in [0.01, 0.1, 1, 10]]
best_lr, tab_lr = scegli_iperparametri(
    "Logistic Regression", lambda **p: LogisticRegression(max_iter=5000,

```

```

random_state=SEED, **p), griglia_lr)
tab_lr

griglia_svm = [{"C": c, "gamma": g} for c in [1, 10, 100] for g in ["scale", 1e-4]]
best_svm, tab_svm = scegli_iperparametri(
    "SVM", lambda **p: SVC(kernel="rbf", random_state=SEED, **p), griglia_svm)
tab_svm

griglia_rf = [{"max_depth": d, "max_features": f} for d in [None, 15] for f in ["sqrt",
0.1]]
best_rf, tab_rf = scegli_iperparametri(
    "Random Forest", lambda **p: RandomForestClassifier(n_estimators=300, n_jobs=-1,
random_state=SEED, **p), griglia_rf)
tab_rf

modelli = {
    "Logistic Regression": LogisticRegression(max_iter=5000, random_state=SEED,
**best_lr),
    "SVM (RBF)": SVC(kernel="rbf", probability=True, random_state=SEED, **best_svm),
    "Random Forest": RandomForestClassifier(n_estimators=300, n_jobs=-1,
random_state=SEED, **best_rf),
}
for nome, modello in modelli.items():
    with warnings.catch_warnings():
        warnings.filterwarnings("ignore", message=".*probability.*deprecated",
category=FutureWarning)
        modello.fit(X_train_s, y_train)
        print(f"{nome} addestrato")

risultati = {}

def valuta(nome, y_vero, y_pred, y_prob):
    metriche = {
        "Accuracy": accuracy_score(y_vero, y_pred),
        "Precision (macro)": precision_score(y_vero, y_pred, average="macro"),
        "Recall (macro)": recall_score(y_vero, y_pred, average="macro"),
        "F1 (macro)": f1_score(y_vero, y_pred, average="macro"),
        "ROC AUC (OvR macro)": roc_auc_score(y_vero, y_prob, multi_class="ovr",
average="macro"),
    }
    risultati[nome] = metriche
    print(f"=== {nome} ===")
    print(classification_report(y_vero, y_pred, target_names=CLASSI, digits=3))
    print(f"ROC AUC (OvR macro): {metriche['ROC AUC (OvR macro)']:.4f}")
    return metriche

def grafici(nome, y_vero, y_pred, y_prob, file):
    fig, ax = plt.subplots(1, 2, figsize=(16, 6.5))

    cm = confusion_matrix(y_vero, y_pred)
    ConfusionMatrixDisplay(cm, display_labels=CLASSI).plot(ax=ax[0], cmap="Blues",
colorbar=False)
    plt.setp(ax[0].get_xticklabels(), rotation=40, ha="right", rotation_mode="anchor")
    ax[0].set_xlabel("Classe predetta")

```

```

ax[0].set_ylabel("Classe reale")
ax[0].set_title(f"{nome} - Matrice di confusione (test)")
ax[0].grid(False)

y_bin = label_binarize(y_vero, classes=range(len(CLASSI)))
# Riquadro ingrandito: le curve sono tutte schiacciate vicino all'angolo in alto a
sinistra
zoom = ax[1].inset_axes([0.35, 0.3, 0.45, 0.45])
curve = [(classe, *roc_curve(y_bin[:, k], y_prob[:, k]))[:2]] for k, classe in
enumerate(CLASSI)]
curve.append(("micro-average", *roc_curve(y_bin.ravel(), y_prob.ravel())[:2]))
for classe, fpr, tpr in curve:
    stile = {"color": "k", "ls": "--", "lw": 2} if classe == "micro-average" else
{"lw": 1.3}
    ax[1].plot(fpr, tpr, label=f"{classe} (AUC = {auc(fpr, tpr):.3f})", **stile)
    zoom.plot(fpr, tpr, **stile)
zoom.set_xlim(0, 0.15)
zoom.set_ylim(0.8, 1.005)
zoom.set_title("zoom", fontsize=8)
zoom.tick_params(labelsize=7)
ax[1].plot([0, 1], [0, 1], color="grey", lw=0.8, ls=":")
ax[1].set_xlabel("False Positive Rate")
ax[1].set_ylabel("True Positive Rate")
ax[1].set_title(f"{nome} - Curve ROC one-vs-rest (test)")
ax[1].legend(loc="lower right", fontsize=8)

salva(file)
plt.show()

SIGLE = {"Logistic Regression": "logreg", "SVM (RBF)": "svm", "Random Forest":
"random_forest"}
for i, (nome, modello) in enumerate(modelli.items()):
    y_pred = modello.predict(X_test_s)
    y_prob = modello.predict_proba(X_test_s)
    valuta(nome, y_test, y_pred, y_prob)
    grafici(nome, y_test, y_pred, y_prob, f"0{4 + i}_{SIGLE[nome]}_cm_roc")

class MLP(nn.Module):
    # MLP per classificazione in K classi; restituisce i logit grezzi
    def __init__(self, input_dim, n_classi, hidden_dims=(256, 128), dropout=0.3):
        super().__init__()
        strati = []
        dim_in = input_dim
        for dim_h in hidden_dims:
            strati += [nn.Linear(dim_in, dim_h), nn.BatchNorm1d(dim_h), nn.ReLU(),
nn.Dropout(dropout)]
            dim_in = dim_h
        strati.append(nn.Linear(dim_in, n_classi))
        self.rete = nn.Sequential(*strati)

    def forward(self, x):
        return self.rete(x)

def tensori(X, y):

```

```

    return torch.tensor(X, dtype=torch.float32), torch.tensor(y, dtype=torch.long)

Xtr_t, ytr_t = tensori(X_train_s, y_train)
Xva_t, yva_t = tensori(X_val_s, y_val)
Xte_t, yte_t = tensori(X_test_s, y_test)

torch.manual_seed(SEED)
loader = DataLoader(TensorDataset(Xtr_t, ytr_t), batch_size=64, shuffle=True,
                    generator=torch.Generator().manual_seed(SEED))

rete = MLP(input_dim=X_train_s.shape[1], n_classi=len(CLASSI))
criterio = nn.CrossEntropyLoss()
ottimizzatore = torch.optim.Adam(rete.parameters(), lr=1e-3)
print(rete)
print("Parametri addestrabili:", sum(p.numel() for p in rete.parameters() if
p.requires_grad))
MAX_EPOCHE, PAZIENZA = 100, 10
storia = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
miglior_loss, epoche_senza_miglioramento, pesi_migliori = float("inf"), 0, None

for epoca in range(1, MAX_EPOCHE + 1):
    rete.train()
    for X_b, y_b in loader:
        ottimizzatore.zero_grad()
        perdita = criterio(rete(X_b), y_b)
        perdita.backward()
        ottimizzatore.step()

    # Valutazione a fine epoca su training e validation (modalità eval: niente dropout)
    rete.eval()
    with torch.no_grad():
        logit_tr, logit_va = rete(Xtr_t), rete(Xva_t)
        storia["train_loss"].append(criterio(logit_tr, ytr_t).item())
        storia["val_loss"].append(criterio(logit_va, yva_t).item())
        storia["train_acc"].append((logit_tr.argmax(1) == ytr_t).float().mean().item())
        storia["val_acc"].append((logit_va.argmax(1) == yva_t).float().mean().item())

    if storia["val_loss"][-1] < miglior_loss:
        miglior_loss, epoche_senza_miglioramento, epoca_migliore = storia["val_loss"]
        [-1], 0, epoca
        pesi_migliori = {k: v.clone() for k, v in rete.state_dict().items()}
    else:
        epoche_senza_miglioramento += 1

    if epoca % 5 == 0 or epoche_senza_miglioramento == PAZIENZA:
        print(f"Epoca {epoca:3d} - loss train {storia['train_loss'][-1]:.4f} - loss val
{storia['val_loss'][-1]:.4f}"
              f" - acc val {storia['val_acc'][-1]:.4f}")
    if epoche_senza_miglioramento == PAZIENZA:
        print(f"Early stopping: nessun miglioramento per {PAZIENZA} epoche")
        break

```

```

rete.load_state_dict(pesi_migliori)
print(f"Ripristinati i pesi dell'epoca {epoca_migliore} (loss di validazione
{miglior_loss:.4f})")

epoche = range(1, len(storia["train_loss"]) + 1)
fig, ax = plt.subplots(1, 2, figsize=(14, 4.5))
ax[0].plot(epoche, storia["train_loss"], label="training")
ax[0].plot(epoche, storia["val_loss"], label="validation")
ax[0].axvline(epoca_migliore, ls="--", color="grey", label=f"epoca migliore
({epoca_migliore})")
ax[0].set_title("MLP - Loss (cross-entropy)")
ax[0].set_xlabel("Epoca")
ax[0].legend()
ax[1].plot(epoche, storia["train_acc"], label="training")
ax[1].plot(epoche, storia["val_acc"], label="validation")
ax[1].axvline(epoca_migliore, ls="--", color="grey")
ax[1].set_title("MLP - Accuracy")
ax[1].set_xlabel("Epoca")
ax[1].legend()
salva("07_mlp_curve_addestramento")
plt.show()

rete.eval()
with torch.no_grad():
    prob_mlp = torch.softmax(rete(Xte_t), dim=1).numpy()
pred_mlp = prob_mlp.argmax(axis=1)

valuta("MLP (PyTorch)", y_test, pred_mlp, prob_mlp)
grafici("MLP (PyTorch)", y_test, pred_mlp, prob_mlp, "08_mlp_cm_roc")

confronto = pd.DataFrame(risultati).T
confronto.round(4)

ax = confronto.plot(kind="bar", figsize=(12, 5), width=0.8, colormap="tab10")
ax.set_ylim(max(0.0, confronto.values.min() - 0.05), 1.0)
ax.set_title("Confronto dei modelli sul test set")
ax.set_ylabel("Valore")
ax.legend(loc="upper left", bbox_to_anchor=(1.01, 1), fontsize=9)
plt.xticks(rotation=0)
salva("09_confronto_modelli")
plt.show()

```